Genetic Programming with Multi-fidelity Surrogates for Large-scale Dynamic Air Traffic Flow Management

Tong Guo, *Graduate Student Member, IEEE,* Yi Mei, *Senior Member, IEEE,* Mengjie Zhang, *Fellow, IEEE,* Ruofei Sun, Yanbo Zhu, *Member, IEEE,* and Wenbo Du, *Member, IEEE*

***Abstract*—Dynamic Air Traffic Flow Management (DATFM) aims at flexibly balancing air traffic demand with limited airspace by scheduling aircraft, particularly during unforeseen events, to maintain efficiency in aviation operations. Genetic Programming (GP) has shown success in evolving effective heuristics across various domains. However, directly adopting GP to DATFM may be less effective due to the computationally intense simulations required for large-scale aircraft decision-making over broad airspace. To address the issue, we develop a novel multi-fidelity surrogate-assisted GP. The core idea is that if a computationally efficient low-fidelity surrogate provides enough information to guide the population toward promising areas effectively, then employing more accurate but resource-intensive evaluations would only increase computational effort without enhancing the direction of evolution. A key innovation in our method is a surrogate management strategy that automatically determines *when* and *which* surrogate model to use, based on collective information from the evolving population. This approach allows for more effective management of computational resources during the evolutionary process, enabling exploration of a broader range of the heuristic space and increasing the likelihood of identifying promising solutions. The proposed method has been tested on various benchmark instances derived from actual air traffic data. Extensive experimental results demonstrate that the proposed algorithm significantly outperforms current state-of-the-art methods in both effectiveness and efficiency.**

***Index Terms*—Genetic programming, evolutionary computation, meta-heuristic, air traffic flow management**

I. Introduction

AIR Traffic Flow Management (ATFM) is key to orchestrating aircraft schedules, determining appropriate flight routes, and allocating time slots to avert congestion and uphold the efficiency of air transport. ATFM confronts challenges arising from the dense scheduling of flights and the dynamic shifts in environmental conditions, driven by unpredictable factors like adverse weather and unforeseen events [1]. As a result, pre-planned schedules may become ineffective, leading to substantial flight delays and reduced transport throughput. Given its critical importance, dynamic air traffic flow management (DATFM) has garnered considerable attention from academia and industry alike. To enhance the capability for reactive scheduling in response to such dynamic situations, several major initiatives have been undertaken, such as NextGen in the U.S. and SESAR in Europe [2].

DATFM is classified as NP-hard [3]. Existing methods to address this complexity fall into two major categories: proactive approaches and reactive heuristics. Proactive approaches, including stochastic programming [4], [5], [6], [7], robust optimization [8], [9], [10], and chance-constraint optimization [11], [12], [13], [14], depend on the precise distribution of uncertain factors, which are often not available in practical scenarios. Additionally, as aircraft scheduling scales up, the computational expense grows exponentially due to the NP-hardness of the problem. In contrast, reactive heuristics involve expert-crafted, manual scheduling rules [15], [16], [17]. Independent of precise knowledge about uncertain distributions, these rules quickly adapt to dynamic environments. Their effectiveness and flexibility have led to widespread adoption in both academic and industry sectors, highlighted by their use in organizations like the National Aeronautics and Space Agency (NASA [18], [19]) and the European Union Aviation Safety Agency (EASA [20], [21]). However, the effectiveness of these heuristics may vary across different scenarios. Moreover, manually designing these heuristics is not only time-consuming but also heavily reliant on domain-specific knowledge.

Genetic Programming (GP) emerges as a promising hyper-heuristic approach for the automatic development of effective scheduling [22], [23] and routing heuristics [24], [25]. The foremost strength of GP is its flexible representation, which facilitates the exploration of a wide range of program structures. Furthermore, the capacity for high interpretability of GP allows for direct analysis of program structures by users [26], which is crucial for real-world applications. Importantly, the heuristics evolved via GP exhibit strong generalizability, rendering them adaptable to diverse scenar-

This work was funded in part by the National Natural Science Foundation of China (NSFC) under Grant U2333218, 52302398, and Supported by Beijing Natural Science Foundation under Grant L241036 *(Corresponding author: Yanbo Zhu and Wenbo Du).*

Tong Guo, Ruofei Sun, and Wenbo Du are with the School of Electronic and Information Engineering, Beihang University, Beijing 100191, China (e-mail: guotong1997@buaa.edu.cn; sunruofei@buaa.edu.cn; wenbodu@buaa.edu.cn).

Yanbo Zhu is with the School of Electronics and Information Engineering, Beihang University, Beijing 100191, China, and also with Aviation Data Communication Corporation, Beijing 100083, China (e-mail: zyb@adcc.com.cn).

Yi Mei and Mengjie Zhang are with the Centre for Data Science and Artificial Intelligence & School of Engineering and Computer Science, Victoria University of Wellington, Wellington, New Zealand (email:yi.mei@ecs.vuw.ac.nz; mengjie.zhang@ecs.vuw.ac.nz).

ios [27]. While the GP framework is universally applicable across various problems, elements such as the terminal set, individual representation, and low-level heuristic template are tailored to specific problem characteristics. To our knowledge, there has been no research exploring the use of GP to automatically evolve suitable heuristics for DATFM.

Simulations are utilized to evaluate a possible GP scheduling rule (an individual). Given that real-world air traffic management often involves large-scale aircraft and waypoints, the exact evaluation of the entire GP population is inherently time-consuming and expensive [28]. Consequently, directly adopting the basic GP framework to DATFM may be less effective due to its limited ability to explore the heuristic space adequately. Simplified simulation-based Surrogate-assisted GP (SaGP) [29], [30], [31], [32] uses partially converged simulations (simulation is paused at a premature stage) to evaluate the population instead of the exact evaluations. With limited time budgets, SaGP employing accurate surrogates can explore broader regions within the heuristic space, thereby increasing the likelihood of identifying more promising individuals.

Existing SaGP methods have limitations from the surrogate utilization. First, existing SaGP methods adopt exact evaluations after surrogates at each generation. However, if the current surrogate could distinguish promising individuals from the population and guide the evolutionary process, switching to a time-intensive exact evaluation at each generation would not enhance the performance but incur additional computation costs. Second, the surrogate fidelity levels of existing SaGP methods are pre-fixed throughout the evolutionary process [30], [31], overlooking the dynamic nature of the evolving population. Specifically, a computationally cheaper low-fidelity surrogate might differentiate individuals effectively at the early stage of evolution. However, as the population progressively converges towards promising regions, distinguishing between individuals with similar phenotypes becomes challenging for the low-fidelity surrogate. On the other hand, consistently using a more accurate high-fidelity surrogate ensures better differentiation but at the expense of considerably higher computational demands and slower simulation speeds, ultimately constraining the exploration of broader regions in the heuristic space. Consequently, existing SaGP methods are unable to strike a balance between surrogate accuracy and simulation computing overhead, and may fall short in efficiently solving large-scale DATFM problems.

Motivated by the above considerations, we develop a novel approach, **Mu**lti-fidelity **S**urrogate-**a**ssisted **GP** (MuSaGP), specifically designed to tackle large-scale DATFM problems efficiently. The overall contributions of this study are listed as follows.

1) To automatically design effective and interpretable reactive heuristics, we developed a GP framework for DATFM, featuring six newly designed problem-tailored terminals and a low-level heuristic template.
2) To enhance the performance of the SaGP methods, we proposed an innovative multi-fidelity surrogate management strategy, which automatically decides *when* and *which* surrogate to employ, guided by progressively accumulated information from the evolving population. This strategy skillfully manages computational resources by dynamically alternating between low-fidelity surrogates and high-fidelity surrogates or exact evaluations, guiding the population to explore effectively and efficiently in the heuristic space.
3) To verify the effectiveness and practicality of the proposed method, we construct a DATFM benchmark instance with different sizes and conditions from real-world air traffic data. A comprehensive experimental study on the real-world dataset demonstrates that MuSaGP significantly outperforms existing state-of-the-art reactive heuristics as well as automated heuristic design methods in terms of air traffic throughput.

The remainder of this paper is organized as follows. First, the background is introduced in Section II. Then, Section III describes the proposed algorithm in detail. Experimental studies and further analysis are described in Sections IV and V, respectively. Finally, Section VI draws the conclusions.

II. Background

A. Problem Description

A simplified representation of an air traffic network $\mathcal{G} = \langle \mathcal{V}, \mathcal{E} \rangle$, encompassing elements airports, waypoints, route segments, and airspace sectors, is depicted in Fig. 1. All related notations that will be used are listed in Table I. The set of nodes $\mathcal{V}$ includes all waypoints and airports, and the set of edges $\mathcal{E}$ consists of air route segments connecting two nodes.

Each edge $e_{u,v} \in \mathcal{E}$ connecting two nodes u, $v \in \mathcal{V}$ has a capacity $C_{u,v}(t)$, limiting the maximum number of aircraft traversing the route segment. In addition, each edge $e_{u,v} \in \mathcal{E}$ has a length denoted by $L_{u,v}$, representing the geographical distance of the route segment connecting waypoints u and $v \in \mathcal{V}$. The airspace is divided into several sectors, with each sector representing a continuous portion of the network and monitored by air traffic controllers. These sectors encompass a subset of nodes and edges. The time $\Delta t_{f,u,v}$ taken by an aircraft $f \in \mathcal{F}$ to travel from node u to v (i.e., traversing edge $e_{u,v}$) depends on the air traffic flow volume $vol_{u,v}(t)$, capacity $C_{u,v}(t)$, and length $L_{u,v}$ of the edge. This relationship is described by the Volume-Delay Function (VDF) [33], which models how travel time is affected by the traffic volume relative to the edge's capacity:

$$\Delta t_{f,u,v} = \texttt{VDF}(1 + \alpha \cdot (\frac{vol_{u,v}(t)}{C_{u,v}(t)})^{\beta}) \cdot \frac{L_{u,v}}{s_f} \qquad (1)$$

where α and β are parameters of the monotonically increasing volume-delay function (VDF), and s_f denotes the free-flow airspeed of aircraft f. $vol_{u,v}(t)$ represents the actual number of aircraft flying along the edge $e_{u,v}$ at time t. Equation (1) suggests that decreases in route capacity $C_{u,v}(t)$ or fluctuations in air traffic volume $vol_{u,v}(t)$ can

TABLE I
NOTATIONS ABOUT THE DATFM PROBLEM.

Notation	Explanation
$\mathcal{V}$	Set of the nodes(airports and waypoints).
$\mathcal{E}$	Set of the edges(route segment)
$\mathcal{F}$	Set of the aircraft
$\mathcal{T}$	Set of time-slot
Ξ	Set of air traffic instances
$\mathcal{N}(u)$	Set of neighbor nodes of node u
$\lvert\mathcal{F}\rvert$	Number of scheduled aircraft
$C_{u,v}(t)$	Capacity of edge $e_{u,v}$ at time t
$\Delta t_{f,u,v}$	Time consumed by aircraft f to traverse edge $e_{u,v}$
$vol_{u,v}(t)$	Air traffic flow volume on edge $e_{u,v}$
$L_{u,v}$	Geographical distance of edge $e_{u,v}$
τ_f	Scheduled take-off time of aircraft f
s_f	Free-flow airspeed of aircraft f
r_f	Flight route of aircraft f
o_f	Scheduled departure airport of aircraft f
d_f	Scheduled arrival airport of aircraft f
T_f	Actual arrival time of aircraft f at arrival airport d_f
$T_{\max}$	Given maximum time limit
$x_{f,u,v,t}$	Decision variable in DATFM
ξ	A given air traffic instance
VDF	Volume-Delay Function

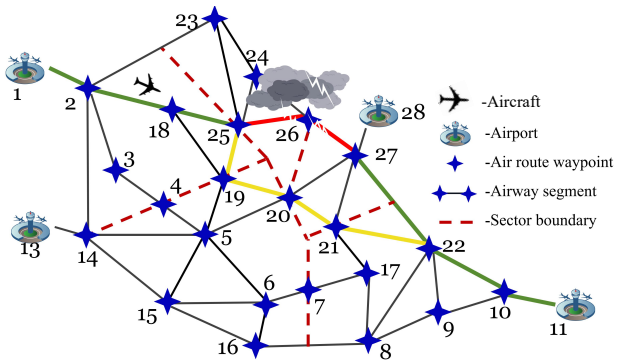

Fig. 1. This is a simple example of a DATFM scenario where an aircraft departs from origin airport 1 and heads towards destination airport 11. The initially planned flight path is $1-2-18-25-26-27-22-10-11$, with the aircraft currently at node 18. Due to adverse weather, the capacity along the route $25-26-27$ (highlighted in red) is significantly reduced, leading to potential congestion and delays. To circumvent this, dynamic rerouting is initiated from node 25, resulting in an adjusted flight route of $1-2-18-25-19-20-21-22-10-11$, effectively avoiding the affected area.

lead to varying levels of congestion, resulting in different degrees of flight delays.

The DATFM contains random variables $vol_{u,v}(t)$ and $C_{u,v}(t)$, each of which can have different sample values. While these random variables can theoretically adhere to specific statistical distributions, their actual distributions in practice are often unknown. To enhance the model's relevance and practicality, we adopt the simulation-based approach to formulate the DATFM. In a DATFM instance *sample*, each traffic volume $vol_{u,v}(t)$ or route capacity $C_{u,v}(t)$ takes a realized value. However, the realized values are unknown until the aircraft arrives at the corresponding edge. The random variables can cause the violation of capacity constraints, known as *route failure*, to occur when the actual traffic volume on a route exceeds its capacity, a situation that compromises flight safety. In this case, the aircraft must either wait for capacity to become available or choose an alternative waypoint to reroute.

The decision variable of DATFM is a 0-1 variable $x_{f,u,v,t}$, which equals 1 if aircraft $f \in \mathcal{F}$ flies from node u to node v at time t; each aircraft f departs from airport o_f at time τ_f heading towards d_f. Given the decision variables $x_{f,u,v,t}$, the flight route of the aircraft $r_f = (o_f, v_{f,1}, v_{f,2}, \cdots, d_f)$ can be obtained by arranging them in the order of time t. Based on this flight route, the time T_f for the aircraft to reach the destination airport d_f can be calculated by computing the time $\Delta t_{u,v}$ taken to traverse each edge and the departure time τ_f:

$$T_f = \tau_f + \sum_{(u,v)\in r_f} \Delta t_{f,u,v} \tag{2}$$

Given a maximum time limit $T_{\max}$, if T_f is less than $T_{\max}$, it implies that aircraft f has completed its schedule. The DATFM model aims to optimize aircraft flight routes, thereby maximizing the expected number of aircraft that complete their schedules within the time period $\mathcal{T} = [0, T_{\max}]$ (i.e., air traffic throughput [34]). This optimization is subject to various constraints and accounts for all the possible values of the random variables after addressing route failures.:

$$\max \quad \mathbb{E}\left[\sum_{f}^{|\mathcal{F}|} \mathbb{I}_\xi \left(T_f \le T_{\max}\right) | \xi \in \Xi\right] \tag{3}$$

$$\text{s.t.} \quad \sum_{f=1}^{|\mathcal{F}|} x^{\xi}_{f,u,v,t} \le C^{\xi}_{u,v}(t),\ \forall (u,v) \in \mathcal{E} \tag{4}$$

$$\sum_{v\in\mathcal{N}(u)} x^{\xi}_{f,u,v,t} \le 1,\ \forall u \in \mathcal{V} \tag{5}$$

$$\sum_{t=0}^{T_{\max}} x^{\xi}_{f,u,v,t} \le 1,\ \forall (u,v) \in \mathcal{E} \tag{6}$$

$$\sum_{v\in\mathcal{V}} x^{\xi}_{f,u,v,t} = \sum_{w\in\mathcal{V}} x^{\xi}_{f,w,u,t},\ \forall u \in \mathcal{V}\setminus\{o_f, d_f\} \tag{7}$$

$$\sum_{v\in\mathcal{V}\setminus\{o_f\}} x^{\xi}_{f,o_f,v,0} = \sum_{u\in\mathcal{V}\setminus\{d_f\}} x^{\xi}_{f,u,d_f,M} = 1 \tag{8}$$

$$x^{\xi}_{f,u,v,t} \in \{0,1\},\ \forall u,v \in \mathcal{V} \tag{9}$$

$$\forall t \in \mathcal{T},\ \forall \xi \in \Xi \text{ for Equation}(4) \tag{10}$$

$$\forall f \in \mathcal{F},\ \forall t \in \mathcal{T},\ \forall \xi \in \Xi \text{ for Equation}(5), (7), (9) \tag{11}$$

$$\forall f \in \mathcal{F},\ \forall \xi \in \Xi \text{ for Equation}(6), (8) \tag{12}$$

Equation (3) delineates the objective function, utilizing the indicator function $\mathbb{I}_\xi(\cdot)$ specific to the instance ξ. It equals one if the conditions in the parentheses are satisfied and zero otherwise. Equation (4) ensures the maximum number of aircraft on each route cannot surpass the capacity limits, which reflects the maximum allowable workload for air traffic controllers. $\xi \in \Xi$ represents an instance *sample*. $C^{\xi}_{u,v}(t)$

contains uncertain factors including the timing, location (i.e., edge $e_{u,v}$), and magnitude of capacity variations. The values of $C^{\xi}_{u,v}(t)$ vary at different timestamps t, which are used to express the uncertain factors of the capacity variation. The values of $C^{\xi}_{u,v}(t)$ are revealed from the instance sample as aircraft arrive at the corresponding edges. Equation (5) mandates that an aircraft departing from node u can only proceed to one of its neighboring nodes $\mathcal{N}(u)$. Equation (6) prohibits aircraft from traversing the same route segments more than once. Equation (7) guarantees the continuity in time and flight trajectory, essential for maintaining a coherent traffic flow. Equation (8) specifies that aircraft f must adhere to its scheduled takeoff and landing at airports o_f and d_f, ensuring compliance with predetermined flight plans. Here, M is a significantly greater value than $T_{\max}$. Equation (9) provides the definition of the variables and sets involved in the model. Equations (10)-(12) define the indexes in the constraints.

B. Related Work

1) Existing Methods for DATFM: The earliest work on DATFM was accomplished by Odoni *et al.* [35]. Since the introduction of the first DATFM model, it has garnered significant attention due to its practicality in real-world air traffic systems, with many subsequent works being proposed in its wake [36]. The existing methods can be roughly divided into two categories, i.e., the proactive methods and the reactive heuristics.

The proactive methods, including stochastic programming [4], [5], [6], [7], robust optimization [8], [9], [10], and chance-constraint optimization [11], [12], [13], [14], usually assume that the uncertain parameters of DATFM can be represented by the known probability distribution. These methods involve transforming the stochastic variables in the model according to the given probability distribution, thereby equivalently converting the uncertainty problem into a deterministic one [1]. This allows for the application of mathematical programming methods such as integer programming and branch-and-bound to find solutions. Under the strict assumption that the given probability distributions hold, these methods can achieve theoretically optimal solutions. However, in actual air traffic systems, obtaining the true probability distributions of these stochastic variables is usually impossible. As a result, these methods often simplify the assumption that these variables follow a Gaussian or uniform distribution [13]. Furthermore, due to the NP-hard nature of the problem, the computational time of mathematical programming solvers can increase dramatically with the size of the problem.

The reactive heuristics do not generate solutions in advance. Instead, they make decisions dynamically and locally in real-time [37]. Not relying on the exact probability distribution of the uncertain factors, reactive heuristics offer the advantage of adaptively responding to unpredicted changes and uncertainties in practical air transport systems [38]. The reactive heuristics have been adopted by organizations, such as NASA [18], [19], EASA [17], [20] and Eurocontrol [21] for use in scheduling aircraft in real-world applications. Reactive heuristics are usually manually crafted by experts, a process that is not only time-consuming but also results in rules that are often too specific, limiting their applicability to a range of different scenarios. For instance, a departure time-slot allocation heuristic grounded in the first-come-first-serve principle is introduced in [20] specifically for DATFM scenarios involving dynamic airport capacity. Meanwhile, dynamic rerouting heuristics, leveraging the D* algorithm [15], [39] and the fast marching tree algorithm [40], have been developed to address scenarios in DATFM where air routes are unexpectedly blocked.

2) Genetic Programming: As a branch of evolutionary algorithms (EAs), GP encodes problem-solving heuristics into individuals. Unlike other EAs, GP, used here as a hyper-heuristic technique, searches within the heuristic space instead of the solution space. It can automatically evolve reactive heuristics by simulating the process of natural selection, where candidate heuristics are iteratively refined and combined to generate increasingly effective heuristics tailored to specific problems. GP has been successfully applied to many combinatorial optimization problems, such as dynamic scheduling [23] and dynamic routing [41], [42], as well as practical applications like search-and-rescue [25] and commuter evacuation [27], [43].

The fundamental structure of GP comprises individual representation with a terminal/function set, a low-level heuristic template, genetic operators, and selection operators. Typically, the first two components are tailored to specific problems, whereas the latter two are broadly applicable across various domains [44]. Individuals are commonly encoded as trees to represent heuristics, where the leaf nodes are the terminal set, reflecting dynamic state variables related to the problem; the root node is the function set, defining how to compute these terminals. The low-level heuristic template acts as a simulator, used to evaluate the quality of an individual [45], [46]. The existing terminals and low-level heuristic templates for scheduling or routing cannot be directly applied to DATFM because of the unique structural and characteristic complexities of air traffic control.

3) Surrogate-assisted GP (SaGP): Evaluating an individual (i.e., a candidate heuristic) of GP may be time-consuming. As a result, given a limited time budget, GP can only explore a very narrow region in the heuristic space, leading to unsatisfactory performance. SaGP is developed by replacing expensive and exact evaluations with faster but less accurate surrogate models. Existing SaGP falls into two distinct categories: *machine learning-based surrogates* and *simplified simulation-based surrogates*, based on the construction of the surrogates. The first category employs existing machine learning models like K-Nearest Neighbors or Kriging model to estimate an individual's fitness by identifying the most similar individual from a pool [47], [48]. The second category utilizes a simplified simulation model, perhaps with a reduced number of jobs and machines,

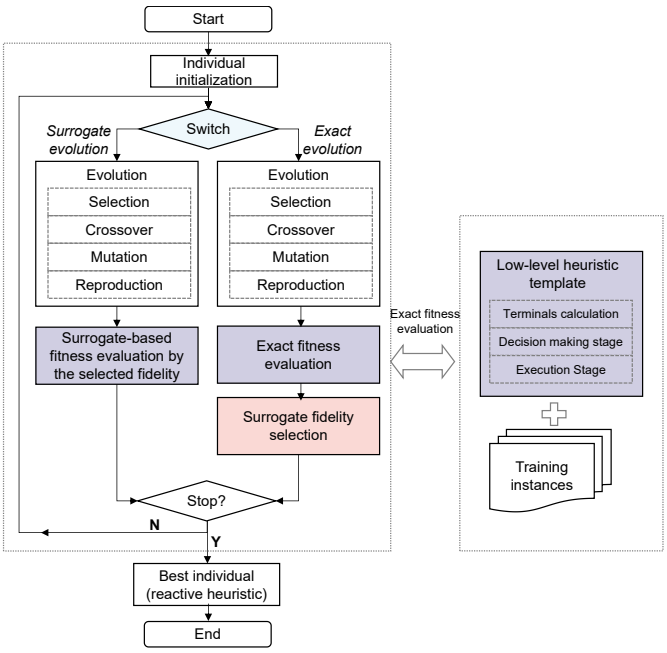

Fig. 2. The flowchart of the proposed MuSaGP algorithm.

as the surrogate [29], [30], [31]. In DATFM, acquiring ample exact evaluation data for surrogate training is challenging due to the high computational cost of simulating large-scale aircraft. In contrast, generating simplified simulations is more feasible, either by truncating the simulation timespan or simulating only a portion of the air traffic network. Consequently, this paper adopts the approach of using simplified simulation-based surrogates.

Most existing SaGP approaches employ a single fixed-fidelity surrogate throughout the entire evolutionary process. Although few studies explored the use of multi-fidelity surrogates in SaGP [30], [31], these approaches usually necessitate manual setting of fidelity levels at different stages or for distinct populations, lacking generalizability across different problems. Existing studies on multi-fidelity EAs [49], [50], [51], [52] are typically designed predominantly for continuous function optimization. These methods primarily investigated how to distinguish the fitness evaluated by low-fidelity machine learning-based surrogates rather than how to determine an appropriate fidelity level for the surrogates. Having different context meanings, they are not applicable to our specific problem domain.

III. Genetic Programming with Multi-fidelity Surrogates

In this section, we present an overview of our newly proposed MuSaGP framework, tailored for DATFM problems. We then delve into a more detailed discussion, outlining the designed heuristic template, the individual representation, and the strategy for managing surrogates.

Algorithm 1: MuSaGP

Input: Training instances Ξ; Population $\mathcal{P}$ with a pool of candidate heuristics
Output: The best evolved reactive heuristic S^*
// Initialization
1 Initialize the population $\mathcal{P}$ by the ramped half-and-half [22];
2 $\mathbf{fit}(\mathcal{P}), \zeta_{\text{fit}}, \zeta_{\text{overhead}} = \texttt{Evaluate}(\mathcal{P}, T_{\max}, \Xi)$;
3 $T^* = \texttt{SelectSurrogate}(\zeta_{\text{fit}}, \zeta_{\text{overhead}})$;
4 Best individual in the population $S^* = \emptyset$, $Flag = False$;
// Evolution and Learning Loop
5 **while** *not termination* **do**
6 **if** g % $RI == 0$ or $Flag$ **then**
 // Evolution with exact evaluation
7 Set the offspring $\mathcal{P}'$ as the top m elites of $\mathcal{P}$;
8 **for** $i = 1 \rightarrow (popsize - m)$ **do**
9 $S' \leftarrow \texttt{Breeding}(\mathcal{P}, \Xi)$;
10 $\mathcal{P}' = \mathcal{P}' \cup S'$;
11 $\mathbf{fit}(\mathcal{P}'), \zeta'_{\text{fit}}, \zeta'_{\text{overhead}} = \texttt{Evaluate}(\mathcal{P}', \overset{\nabla}{T_{\max}}, \Xi)$;
12 Update $T^* = \texttt{SelectSurrogate}(\zeta'_{\text{fit}}, \zeta'_{\text{overhead}})$;
13 Update $\mathcal{P} \leftarrow \mathcal{P}'$;
14 $Flag = False$;
15 **else**
 // Evolution with surrogate evaluation
16 Set the offspring $\mathcal{P}'$ as the top m elites of $\mathcal{P}$;
17 **for** $i = 1 \rightarrow (n \cdot popsize - m)$ **do**
18 $S' \leftarrow \texttt{Breeding}(\mathcal{P}, \Xi)$;
19 $\mathcal{P}' = \mathcal{P}' \cup S'$;
20 $\mathbf{fit}(\mathcal{P}'), \zeta'_{\text{fit}}, \zeta'_{\text{overhead}} = \texttt{Evaluate}(\mathcal{P}', \overset{\nabla}{T^*}, \Xi)$;
21 $Flag = \texttt{StagDetect}(\mathbf{fit}(\mathcal{P}'), \mathbf{fit}(\mathcal{P}))$;
22 Update $\mathcal{P}$ as the top $popsize$ individuals in $\mathcal{P}'$;
23 Update $g \leftarrow g + 1$;
24 Update $S^* = \texttt{argmax}_{S \in \mathcal{P}}\{\cdots fit(S) \cdots\}$;
25 **return** S^*;

A. Overall Framework

Fig. 2 shows the overall framework of MuSaGP and its detailed procedures are described in Algorithm 1.

1) Population Initialization : Each individual in the population represents a possible reactive heuristic, describing how to schedule the aircraft within the air traffic network. The individuals are represented using a standard tree structure, comprising terminals for leaf nodes and functions for non-leaf nodes. During the initialization phase (lines 1-4), the population $\mathcal{P}$ is initialized using the ramped half-and-half strategy, a technique renowned for boosting initial population diversity [22]. Then, the population undergoes evaluation against the training instances Ξ over a complete simulation timespan $T_{\max}$, generating the fitness vector $\mathbf{fit}(\mathcal{P}) \in \mathbb{R}^{popsize}$.

2) Individual Evaluation: Each individual in the population represents a potential reactive heuristic. These heuristics are essentially algorithms or rules that construct flight routes on the fly for each aircraft as new capacity information becomes available. The evaluation of individuals in GP involves assessing how well these heuristics perform in solving the DATFM problem.

The pseudocode for fitness evaluation on the training instance Ξ is outlined in Algorithm 2. In line 5, each individual (reactive heuristic) is applied to each instance $\xi \in \Xi$ using the proposed simulation model (i.e., `LowHeuristic`,

Algorithm 2: $\langle \mathbf{fit}(\mathcal{P}), \zeta_{\text{fit}}, \zeta_{\text{overhead}} \rangle \leftarrow \texttt{Evaluate}(\mathcal{P}, T_{\text{sim}}, \Xi)$

Input: Population $\mathcal{P}$, simulation timespan T_{sim}, training instances Ξ;
Output: Population fitness $\mathbf{fit}(\mathcal{P})$, multi-fidelity fitness matrix ζ_{fit}, multi-fidelity time cost matrix ζ_{overhead}

1 Initialize population fitness tensor $\mathbf{y}_{\text{fit}} \in \mathbb{R}^{popsize \times |\Xi| \times \dim}$;
2 Initialize simulation time cost tensor $\mathbf{y}_{\text{time}} \in \mathbb{R}^{popsize \times |\Xi| \times \dim}$;
3 **for** *individual and individual index* $\langle S, i \rangle \in \mathcal{P}$ **do**
4 **for** *instance and instance index* $\langle \xi, j \rangle \in \Xi$ **do**
5 $\mathbf{y}_{\text{fit}}[i, j, :], \mathbf{y}_{\text{time}}[i, j, :] = \texttt{LowHeuristic}(S, T_{\text{sim}}, \xi)$;
 // Element-wise averaging of matrices over the instance dimension
6 $\zeta_{\text{fit}} = (\sum_{j=1}^{|\Xi|} \mathbf{y}_{\text{fit}}[:, j, :])/|\Xi|$;
7 $\zeta_{\text{overhead}} = (\sum_{j=1}^{|\Xi|} \mathbf{y}_{\text{time}}[:, j, :])/|\Xi|$;
8 $\mathbf{fit}(\mathcal{P}) = \zeta_{\text{fit}}[:, -1]$;
9 **return** $\langle \mathbf{fit}(\mathcal{P}), \zeta_{\text{fit}}, \zeta_{\text{overhead}} \rangle$;

Algorithm 3: $S' \leftarrow \texttt{Breeding}(\mathcal{P}, \Xi)$

Input: Population $\mathcal{P}$; Training instances Ξ
Output: One individual of offspring S'

1 $rm = \texttt{random}(0, 1)$;
2 **if** $rm < p_b$ **then**
 // Crossover
3 $S_1, S_2 = \texttt{LexicaseSelection}(\mathcal{P}, \Xi)$;
4 $S' = \texttt{crossover}(S_1, S_2)$;
5 **else if** $rm < (p_b + p_m)$ **then**
 // Mutation
6 $S' = \texttt{mutation}(\texttt{LexicasSelection}(\mathcal{P}, \Xi))$;
7 **else**
 // Reproduction
8 $S' = \texttt{copy}(\texttt{LexicaseSelection}(\mathcal{P}, \Xi))$;
9 **return** S';

Algorithm 4: $Flag \leftarrow \texttt{StagDetect}(\mathbf{fit}(\mathcal{P}'), \mathbf{fit}(\mathcal{P}))$

Input: Phenotypic data of the population from two consecutive generations of surrogate-evolution $\mathbf{fit}(\mathcal{P}'), \mathbf{fit}(\mathcal{P})$;
Output: $Flag$ indicates whether to switch to exact evolution;

1 $\hat{\mathbf{fit}}(\mathcal{P}') \leftarrow \texttt{Sort}(\mathbf{fit}(\mathcal{P}'))$;
2 $\hat{\mathbf{fit}}(\mathcal{P}) \leftarrow \texttt{Sort}(\mathbf{fit}(\mathcal{P}))$;
3 $\mathscr{E}' = \texttt{SelectElite}(\mathcal{P}', m)$;
4 $\mathscr{E} = \texttt{SelectElite}(\mathcal{P}, m)$;
5 $Flag \leftarrow \texttt{SignedRankTest}(\mathscr{E}', \mathscr{E}, \overline{p})$;
6 **return** $Flag$

elaborated in Section III-C). This simulation utilizes the individuals (reactive heuristics) to make decisions at the current waypoint by calculating the priority of adjacent waypoints according to the network's dynamic states. Then, the waypoint with the highest priority is selected as the next point the aircraft fly to. In this manner, reactive heuristics act like constructive heuristics, which construct the flight routes for aircraft online. Finally, utilizing the flight routes constructed by the reactive heuristic on each instance, the objective value for each instance can be calculated based on the number of aircraft that successfully complete their schedules within the given timeframe. The fitness of an individual is then calculated as the average of these objective values across all training instances. An individual with higher fitness indicates that the corresponding reactive heuristic is more effective at scheduling the aircraft, thereby improving the throughput of the air traffic network.

3) Evolution with Exact Evaluation: During lines 6-14, the population undergoes an evolution phase where each individual is evaluated using the complete simulation timespan $T_{\max}$. The breeding stage involves the application of genetic operators by the selected parents to produce offspring, which is elaborated in Algorithm 3. To improve the population diversity, lexicase selection is employed to select suitable parents from the population [53]. MuSaGP incorporates tree-swapping crossover and subtree mutation methods for this purpose [54]. As depicted in Fig. 3, for tree swapping crossover, one tree is processed by randomly selecting a subtree from each parent and exchanging them, while the other tree is swapped in its entirety. Subtree mutation involves generating a new subtree from a combination of terminals and functions, which then replaces a randomly selected subtree in the parent, introducing new genetic variations.

4) Surrogates Creation: In lines 11-12, the population is evaluated on training instances with the complete simulation timespan $T_{\max}$, to calculate its fitness. For surrogate creation, we employ partially converged simulations, allowing us to generate surrogates with varying fidelity. The fidelity here is dependent on the simulation timespan: longer simulations yield more accurate but time-intensive surrogates, whereas shorter simulations offer less precision but are computationally more efficient. Throughout the simulation process, we record the population fitness and the amount of computational resources consumed by the simulation (quantified in terms of CPU runtime) at different K checkpoints (i.e., ζ_{fit} and ζ_{overhead}). These records are crucial for the surrogate selection in line 12 (`SelectSurrogate`, elaborated in Section-III.D), where we determine the best surrogate in terms of accuracy and computational overhead. In other words, `SelectSurrogate` determines *which* surrogate should be used to evolve the population.

5) Evolution with Surrogates: In lines 16-22, the evolution of the population leverages the selected surrogate in conjunction with the brood recombination strategy [55]. This phase maintains the same parent selection, genetic operators, and environmental selection as the evolution involving exact evaluation. During breeding, we employ the brood recombination strategy to produce a substantial number of offspring ($n \cdot popsize$). These offspring are then evaluated using the surrogates, which necessitate significantly less simulation timespan ($T^* \ll T_{\max}$). Thanks to the cheaper evaluation of surrogates, the integration of brood recombination and surrogate evaluation enables the population to explore a broader range of the heuristic space, thereby enhancing the likelihood of discovering superior reactive heuristics.

6) Adaptive Switching Strategy: Unlike existing SaGP frameworks, where both surrogate evaluation and exact evaluation are conducted at each generation, MuSaGP consistently employs surrogate evaluations over multiple consecutive generations. The core idea here is that if surrogates are capable of accurately identifying promising individuals from

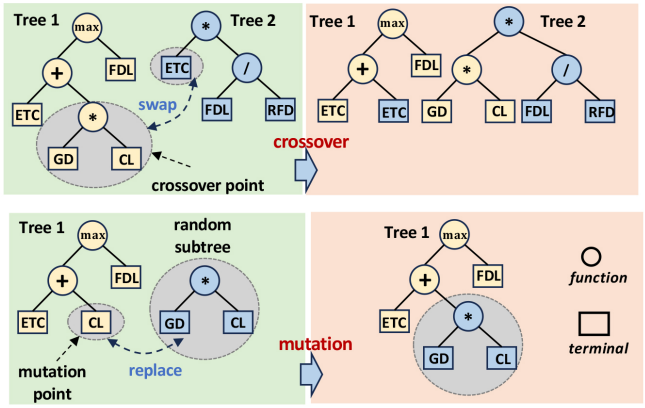

Fig. 3. Schematic description of the tree swapping crossover and the subtree mutation.

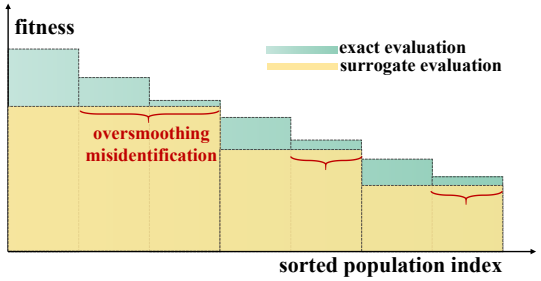

Fig. 4. Schematic description of oversmooth phenomenon within surrogate-evaluated population.

a large intermediate population, then conducting resource-intensive exact evaluations becomes unnecessary, thereby saving computational resources and enabling more efficient exploration of the search space. Conversely, if surrogates fail to distinguish promising candidates, i.e., over-smoothing the fitness of the population, then a shift to exact evaluations is crucial to update the fidelity level of the used surrogate based on the latest population information. Fig. 4 depicts the oversmooth phenomenon. If the surrogate resolution is inadequate, it may lead to incorrect selection of parents and misidentification of promising individuals due to loss of selection pressure on the population, diverting the search in unproductive directions and potentially causing population stagnation.

Motivated by this, a switching trigger mechanism based on significance tests has been devised that focuses on detecting stagnation: if the population shows no improvement over two consecutive generations, with this trend confirmed at a high confidence level, it suggests stagnation. This triggers a shift to exact evaluation methods. Specifically, we use the Wilcoxon Signed-Rank Test [56] to assess statistically significant improvements across generations. The detailed procedures are presented in Algorithm 4. The significance test is conducted on the sorted fitness values of m elites from two consecutive generations in surrogate evolution. Note that the Wilcoxon Signed-Rank Test is tailored for paired or related samples. In our approach, we matched the ranking positions among the elite set rather than directly pairing individuals across generations since the individuals might not represent the same individuals in different generations. We proceed with the assumption that each rank within the population undergoes a 'paired' modification in relation to its predecessor. Given the confidence level, a significant improvement suggests the used surrogate can effectively guide the search for better solutions. Conversely, if these conditions are not met, it indicates difficulty in finding better solutions even with a brood recombination strategy, indicating a need to switch to exact evaluation for surrogate correction.

B. Individual Representation

In our study, each individual symbolizes a low-level heuristic for aircraft scheduling, characterized as a tree-based priority function. This priority function is pivotal in determining the most appropriate waypoint for an aircraft's progression. Structurally, an individual is composed of terminals and functions. The careful design of terminals is vital for the success of GP [22]. Terminals should encompass as many facets of the problem domain as possible, ensuring a comprehensive representation. However, it is crucial to avoid an excessive number of terminals to prevent creating an overly large heuristic space. In our work, we have devised six terminals by extracting the features of waypoints and aircraft within the DATFM context. These terminals are detailed in Table II, with the first two associated with aircraft and the last four with waypoints.

Fig. 3 illustrates an example individual (reactive heuristic) represented as a tree. This tree structure can be converted into a mathematical function, which assigns priority to waypoints for each aircraft. These terminals vary dynamically with different timestamps, allowing the reactive heuristics to adapt in real time. This means that the algorithm makes decisions based on the current system state at each moment, effectively handling capacity uncertainties by rerouting aircraft in response to evolving conditions.

C. Designed Low-level Heuristic

Given an individual $S \in \mathcal{P}$ representing a reactive heuristic, the heuristic template generates a flight plan by applying S to a specific instance $\xi \in \Xi$. This flight plan is a variable-length sequence of events, each event being a tuple $(f, t, v_i, v_j, \Delta t_{f,i,j})$. This tuple signifies that aircraft f moves from waypoint v_i to v_j at time t, with $\Delta t_{f,i,j}$ representing the expected flight time for this segment. The flight time $\Delta t_{f,i,j}$ is calculated using the volume-delay function that considers the geographical distance of the route segment and the density of traffic flow. The overall process of the heuristic template is detailed in Algorithm 5, which includes both decision-making and execution stages.

TABLE II
TERNIMAL SET OF MUSAGP IN DATFM

Notation	Description
RFD	Remaining flight distance to destination airport d_f
ETC	Estimated time cost for remaining flight distance
CL	Capacity of route segment to waypoint v_i
FDL	Flow density of route segment to waypoint v_i
GD	Geographical distance to waypoint v_i
NUM	Number of aircraft that have been assigned to waypoint v_i

1) Decision-making Stage: Lines 5-18 outline the decision-making stage of generating flight plans for aircraft. Initially, all aircraft that have departed from their origin airports are identified (lines 6-7). Two scenarios are then considered.

i) *Aircraft at Destination (lines 8-10).* If an aircraft's current node v_f^c matches its destination airport d_f, it implies arrival. Here, the arrival time T_f is set to the current timestamp t, and the aircraft f is removed from the set of active aircraft.

ii) *Aircraft En-Route (lines 12-18).* For aircraft yet to reach their destination, the next route segment must be determined. This begins by identifying all potential neighboring nodes of the current node (i.e., $\mathcal{N}(v_f^c)$). For each neighboring node $v_i \in \mathcal{N}(v_f^c)$, terminals $\chi_{i,f}$ are derived based on the status of aircraft and the node v_i. The function $\Gamma(\cdot)$ then evaluates these terminals and the tree structure of the individual S, assigning a priority p_{v_i} to each node. The node with the highest priority, denoted as v_f^{c*}, is subsequently incorporated into the flight plan.

The reactive heuristics construct the flight routes for aircraft online. When unpredictable events reduce the capacity of certain routes, the heuristic can immediately adjust its decisions by redirecting aircraft to less affected routes. The reactive nature of the GP-evolved heuristics allows them to dynamically respond and adapt to these uncertain changes as they happen.

2) Execution Stage: It should be noted that in some cases, at some waypoints, multiple aircraft may exist to be scheduled at the same time. The simulation is conducted in an asynchronous manner to replicate real-world conditions, meaning the status of the air traffic network will be immediately updated as each aircraft's flight plan is implemented. Therefore, the performance of the generated execution plan can be affected by the order of the decisions made for the active aircraft. To manage this scenario, MuSaGP adopts a prioritization strategy where the flight plan for the aircraft with the minimal remaining flight time is executed first. This approach aligns with the objective to maximize the number of aircraft completing their schedules within a specified time frame. Implementing this strategy not only potentially increases the number of scheduled aircraft but also adheres to the first-come and first-served principle commonly observed in real-world air traffic management.

Algorithm 5: $\langle$**Fit**, **Time**$\rangle \leftarrow$ `LowHeuristic`(S, T_{sim}, ξ)

Input: Individual S, simulation timespan T_{sim}, an instance ξ;
Output: Throughput and simulation time cost $\langle$**Fit**, **Time**$\rangle$
1 Read $\mathcal{G}_\xi = \langle \mathcal{V}_\xi, \mathcal{E}_\xi \rangle, \mathcal{F}$ from the given instance ξ;
2 **for** *aircraft* $f \in \mathcal{F}$ **do** Read **Plan**$_f$ and current node v_f^c ;
3 Initialize **Fit**=$\emptyset$, **Time**=$\emptyset$, $t = 0$, CPU time $t_0 =$ `CPUTime`;
`// Navigate aircraft based on node priority`
4 **while** $t \leq T_{sim}$ **do**
5 **for** *aircraft* $f \in \mathcal{F}$ **do**
6 **if** $\tau_f \geq t$ **then**
7 **continue**;
8 **if** $v_f^c = d_f$ **then**
9 $T_f = t$;
10 $\mathcal{F} = \mathcal{F} \setminus \{f\}$;
11 **else**
12 Get neighbors set $\mathcal{N}(v_f^c)$ of v_f^c by $\mathcal{G}_\xi$;
13 **for** *each* $v_i \in \mathcal{N}(v_f^c)$ **do**
14 Get the terminal sets $\chi_{i,f}$ of v_i and f by $\mathcal{G}_\xi$ at time t;
15 Calculate the priority value $p_{v_i} = \Gamma(\chi_{i,f}, S)$;
16 $v_f^{c*} =$ `argmax`$_{v_i \in \mathcal{N}(v_i)} p_{v_i}$;
17 **Plan**$_f$ = **Plan**$_f \cup \{(v_f^c, v_f^{c*}, t)\}$;
18 $v_f^c = v_f^{c*}$;
19 Calculate throughput $y_\xi = \sum_{f=1}^{|\mathcal{F}|} \mathbb{I}_\xi \left(T_f \leq T_{\max}\right)$;
20 Calculate simulation time cost $t_{\text{cost}} =$ `CPUTime` $- t_0$;
`// Record at each breakpoint of exact evaluation`
21 **if** $t \ \% \ \Delta\tau = 0$ **then**
22 Update **Fit** = **Fit** $\cup\{y_\xi\}$, **Time** = **Time** $\cup\{t_{\text{cost}}\}$;
23 $t = t + 1$;
24 **return** $\langle$**Fit**, **Time**$\rangle$

Algorithm 6: $T^* \leftarrow$ `SelectSurrogate`$(\zeta_{\text{fit}}, \zeta_{\text{overhead}})$

Input: Multi-fidelity fitness matrix ζ_{fit}, simulation time cost matrix ζ_{overhead};
Output: Best trade-off simulation timespan T^* ;
1 Initialize set trading off $accuracy$ and simulation $overhead$ of different fidelity simulations **Tradeoff** $= \emptyset$;
2 **for** $k = 1 \rightarrow K$ **do**
3 $runtime_k = \sum_{i=1}^{popsize} (\zeta_{\text{overhead}}[i,k])/popsize$;
4 $acc_k =$ `SpearmanCorr`$(\zeta_{\text{fit}}[:,k], \zeta_{\text{fit}}[:,-1])$;
5 **Tradeoff** $=$ **Tradeoff** $\cup \{(runtime_k, acc_k)\}$;
6 **Tradeoff** $\leftarrow$ `ndSorting`(**Tradeoff**);
7 **for** $i = 1 \rightarrow |$**Tradeoff**$|$ **do**
8 Calculate reflex angle θ_i based on Equation (11) ;
9 $T^* =$ `argmax`$_{T \in [1, T_{\max}]} \{\theta_i | i \in [1, |$**Tradeoff**$|]\}$;
10 **return** T^*;

D. Multi-fidelity Surrogates Management

A simulation for an individual spans a complete time frame $T_{\max}$. The partially converged simulation is adopted to create surrogates [29]. Specifically, as outlined in Algorithm 5 (lines 21-23), we introduce K uniformly distributed checkpoints along this timeline, creating K surrogates that vary in fidelity and simulation overhead (measured by CPU runtime). Given the time interval $\Delta\tau$, K is determined by $(T_{\max}/\Delta\tau - 1)$. This process, applied across the entire population, results in matrices ζ_{fit} and ζ_{overhead} that capture multi-fidelity fitness and computational overhead of simulations, respectively. Each element (i, k) within these matrices

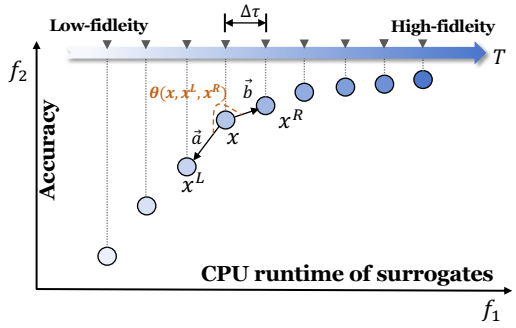

Fig. 5. Illustration of selecting the surrogate.

records the objective value and required CPU runtime for the ith individual using the kth surrogate, where the simulation stops at time $k \cdot T_{\max}/(K+1)$. The final column of ζ_{fit} represents the true objective value obtained from the exact evaluation.

1) Accuracy-Overhead Trade-offs in Surrogates: The simulation overhead for the kth surrogate is determined by averaging the corresponding column in ζ_{overhead} (see line 3 of Algorithm 6). To estimate the accuracy of surrogates, i.e., their ability to preserve the rank order of individuals as compared with exact evaluations, we adopt Spearman correlation. This choice allows for a direct comparison between rankings from surrogate-based evaluations and those from exact evaluations. Spearman correlation, a non-parametric measure, is selected over Pearson correlation due to its ability to evaluate the monotonic relationship between surrogates and actual evaluations, focusing on the rank order rather than the numerical values. Additionally, Pearson correlation requires the assumption of a normal distribution and assesses the linear relationship between variables, conditions that cannot always be met. Specifically, the accuracy of the kth surrogate is given by:

$$acc_k = \texttt{SpearmanCorr}(\zeta_{\text{fit}}[:,k], \zeta_{\text{fit}}[:,-1]) \tag{13}$$

The equation of `SpearmanCorr` is given in [57]. In the fitness matrix ζ_{fit}, the kth column, denoted by $\zeta_{\text{fit}}[:,k]$, represents the fitness values of the population as evaluated by the kth surrogate model. The last column, indicated by $\zeta_{\text{fit}}[:,-1]$, corresponds to the fitness values determined through exact evaluations of the population. A higher acc_k value signifies greater surrogate accuracy. Ideally, if individual a outperforms b in precise evaluations, an accurate surrogate should reflect this ranking order.

2) Surrogate Selection: In managing the trade-off between simulation overhead and accuracy among surrogates of varying fidelities, we aim to select a surrogate that balances these aspects. The selection criterion is the *knee point* within the trade-off set, which is identified as the point with the maximal reflex angle relative to its adjacent points [58]. The knee point surrogate is then used for population evolution. To locate the knee point, we first conduct non-dominated sorting [59] for the trade-off set and employ an angle-based methodology. As illustrated in Fig. 5, for any point x in the trade-off set, represented as $\mathbf{Tradeoff}$, the reflex angle $\theta(x, x^L, x^R)$ is calculated using equation (14) (spans from π to 2π) when x is concave; otherwise, it is assigned a substantially smaller value. A point is considered concave if it lies above the line segment connecting its left and right neighbors.

$$\theta(x, x^L, x^R) = \arccos\left(\frac{\langle \vec{a}, \vec{b}\rangle}{\|\vec{a}\| \cdot \|\vec{b}\|}\right) \tag{14}$$

where $\vec{a} = [f_1(x^L) - f_1(x), f_2(x^L) - f_2(x)]$ and $\vec{b} = [f_1(x^R) - f_1(x), f_2(x^R) - f_2(x)]$. Here, $\langle \vec{a}, \vec{b}\rangle$ represents the inner product, and $\|\vec{a}\|$ signifies the norm of vector $\vec{a}$. The knee point is thus ascertained as the point with the largest $\theta(x, x^L, x^R)$ value that denotes the most pronounced trade-off between accuracy and simulation overhead.

E. Computational Complexity Analysis

After being trained on the training dataset, MuSaGP generates the best individual, which serves as a candidate reactive heuristic for scheduling aircraft during the testing phase. The computational cost during the testing phase is influenced by the calculation of terminals. The calculation of these six terminals is achieved in nearly constant time. Additionally, determining the next node for each aircraft, which depends on a limited number of neighbors per node, is also a constant time operation. Therefore, the primary determinants of MuSaGP's time complexity are the total number of aircraft ($|\mathcal{F}|$) and the average number of nodes traversed by each aircraft ($|\mathcal{V}|$). Under the assumption that aircraft do not revisit nodes, and considering the worst-case scenario where each aircraft visits every node once, the time complexity stands at $O(|\mathcal{F}| \cdot |\mathcal{V}|)$. In practical scenarios, given the relative constancy of $|\mathcal{V}|$, the time complexity of MuSaGP during the testing phase is nearly linear relative to the number of aircraft. This makes MuSaGP well-suited for real-time scheduling in large-scale DATFM problems.

IV. Experimental Studies

In this section, we first describe the used DATFM benchmark. Then, we give the experimental settings. Finally, we present the comparison results and discussions.

A. Datasets and Instance Generation

As a complex practical engineering problem, there is no public benchmark for DATFM. Therefore, we have established a comprehensive benchmark derived from actual air traffic data within Chinese domestic airspace. The original data, provided by Aviation Data Communication Corporation, records daily air traffic operations in history, where each data provides spatial topology of the air traffic network, airspace sector spatial structure, meteorological radar echo maps, weather messages, actual flight plans, and the capacity limits of airway segments. We focus on the dynamic fluctuations in airway segment capacity caused

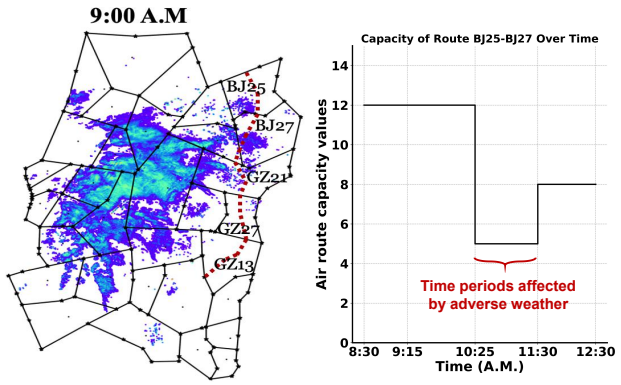

Fig. 6. It features an example of an impacted air route (highlighted in red) spanning five sectors (BJ25, BJ27, GZ21, GZ27, GZ13). The right figure illustrates the capacity variation of air route BJ25-BJ27 over time.

by adverse weather conditions, which are considered the most important and often unpredictable factors [1], [60]. We directly extract the dynamic capacity $C_{u,v}(t)$ from the historical data. Fig. 6 illustrates an air route impacted by adverse weather conditions along with the corresponding dynamic capacity values over time.

Based on these definitions, we extracted 58 instances from the original data to create this benchmark. Thirty of them were produced between May and August 2022, serving as the training set for GP; the remaining 28 were collected between May and June 2023, serving as the test set. These instances are further classified into large ($|\mathcal{F}| > 3000$), medium ($500 < |\mathcal{F}| < 2100$), and small scales ($|\mathcal{F}| < 500$), according to the number of aircraft affected by adverse weather conditions. Each instance is named '*weather-scale*.' For instance, '*mild-m1*' denotes an instance with mild weather conditions and a medium-size scale. The maximum scheduling timespan $T_{\max}$ is established to span an entire day, which is equivalent to 1440 minutes. The time interval for the checkpoints $\Delta\tau$ is set at 5 minutes. Consequently, this configuration results in the creation of 287 surrogates, each with varying fidelity, as calculated by $(\frac{1440}{5} - 1)$.

B. Experiment Settings

To verify the effectiveness of the proposed MuSaGP, we design two sets of experiments.

1) Compared with the state-of-the-art reactive heuristics for DATFM, to verify the quality of the scheduling policy evolved by MuSaGP.
2) Compared with the state-of-the-art automated scheduling heuristics obtained by GP and reinforcement learning (RL), to verify the superiority of MuSaGP over the advanced automated heuristic design frameworks.

1) Compared Reactive Heuristics: Five state-of-the-art reactive heuristics for dynamic traffic scheduling problems and the actual flight schedules are selected as baselines. Specifically, these include CCRP [16], D* Lite [15], CASPER [61], and MUEGF [62], four recently developed heuristics for capacity-constrained routing and scheduling that have demonstrated competitive performance. Additionally, CASA [17], the standard computer-assisted slot allocation algorithm for DATFM which is a rule-based heuristic method frequently employed in actual operations by Eurocontrol [21] is also compared. Moreover, the actual schedules assigned by air traffic controllers for the corresponding day are also used as baselines (referred to as REAL), reflecting the expertise and decision-making skills of human professionals in the field. The exact approaches are not applicable to the dynamic problem here due to the unknown distribution of the uncertain factors. Therefore, they are not selected in the comparison.

2) Compared Automated Scheduling Heuristics: Three state-of-the-art surrogate GP methods are selected as baselines including GP with adaptive surrogates (ASGP) [31], GP with generation-range surrogates (GSGP) [31], and multi-population GP with collaborative multi-fidelity surrogates ($\mathrm{M^3GP_2}$) [30]. Moreover, a competitive deep reinforcement learning-based automated scheduling heuristics, i.e., RL-informed analytics (RLIPA), is also compared [63].

3) Parameter Setting: As for the proposed MuSaGP, the parameter settings are given in Table III. The proposed MuSaGP has three unique parameters m, RI, and $\overline{\rho}$. m represents the number of elite individuals used to detect whether the population has stagnation issues. Since the detection relies on the signed-rank test, m is set to 30 to ensure a reliable statistical evaluation [64]. RI controls the maximum generations used for each surrogate-based evolution. A lower RI value signifies a more frequent switching strategy. $\overline{\rho}$ determines the sensitivity to switching strategies. Specifically, a lower $\overline{\rho}$ value increases the likelihood of a switch occurring. We tune the two parameters $(RI, \overline{\rho})$ based on empirical studies. The full ablation results can be found in Fig. S1 of the supplementary material. Based on the results, the setting $(RI, \overline{\rho}) = (10, 0.20)$ achieves the best performance. The other parameters of MuSaGP are common settings frequently used in existing GP methods [30], [55], which have been showing their effectiveness in dynamic scheduling problems. In order to make the comparisons as fair as possible, the same CPU runtime is given to all the compared GP and RL methods. The maximal runtime is set to 4320 minutes. This given time budget results in a maximum of approximately 100 to 120 generations. The preliminary experiments show that all the compared GP

TABLE III
PARAMETER SETTING OF MUSAGP.

Name	Value
Elite size m	30 [64]
RI	10
Confidence level $\overline{\rho}$	0.20
Population size	500 [55]
size of intermediate population	500 × 3 [55]
Crossover/Mutation/Reproduction rate	80%/15%/5% [30]
Initial minimum/maximum depth	2/6 [30]
Maximal depth of programs	8 [30]
function set	{+; -; *; /; max; min } [47]

TABLE IV
MEAN, STANDARD DEVIATION, AND SIGNIFICANCE TEST OF THE 30 RUNS OF THE COMPARED ALGORITHMS ON THE **OBJECTIVE VALUES**. HIGHER VALUES INDICATE BETTER ALGORITHM PERFORMANCE. THE BEST RESULTS FOR EACH INSTANCE ARE HIGHLIGHTED IN BOLD. 9 OUT OF 28 TEST INSTANCES ARE PRESENTED. THE FULL RESULTS CAN BE FOUND IN THE SUPPLEMENTARY MATERIAL

Instance	D* Lite	CCRP	CASPER	MUEGF	CASA	REAL	ASGP	GSGP	M^3GP_2	RLIPA	MuSaGP
mild-s1	267(+)	464(=)	**466**(=)	463(=)	464(=)	396(+)	464.9(2)(=)	465.2(1)(=)	463.8(1)(=)	465.0(2)(=)	464.6(2)
mild-m1	1876(+)	1877(+)	1611(+)	1298(+)	1877(+)	1544(+)	1975.1(28)(+)	1975.6(21)(=)	1971.3(38)(=)	1986.3(18)(=)	**2002.0**(35)
mild-l1	3411(+)	3154(+)	2746(+)	2284(+)	3154(+)	2652(+)	3391.3(66)(+)	3397.1(36)(+)	3381.5(70)(+)	3422.1(37)(=)	**3448.2**(45)
moderate-s1	203(+)	396(+)	353(+)	371(+)	396(+)	341(+)	395.9(2)(+)	396.2(1)(+)	397.7(1)(=)	396.3(2)(+)	**399.3**(2)
moderate-m1	1940(+)	1919(+)	1696(+)	1366(+)	1919(+)	1583(+)	2009.3(31)(+)	2012.8(25)(+)	2006.3(36)(+)	2028.2(25)(=)	**2040.9**(27)
moderate-l1	3548(+)	3283(+)	2981(+)	2511(+)	3283(+)	2764(+)	3537.0(62)(=)	3541.2(36)(=)	3530.0(69)(=)	3559.5(36)(=)	**3578.2**(40)
severe-s1	199(+)	380(+)	335(+)	356(+)	380(+)	332(+)	383.4(2)(+)	383.2(2)(+)	384.9(3)(+)	385.3(3)(+)	**388.9**(3)
severe-m1	1557(+)	1579(+)	1147(+)	984(+)	1579(+)	1331(+)	1623.6(28)(+)	1632.4(36)(=)	1629.7(22)(=)	1635.4(38)(=)	**1657.3**(43)
severe-l1	2174(+)	2162(+)	2038(+)	1793(+)	2158(+)	1755(+)	2270.0(32)(+)	2270.8(24)(+)	2264.4(42)(=)	2282.7(25)(=)	**2296.3**(23)
W/T/L	19/9/0	27/1/0	27/1/0	27/1/0	27/1/0	28/0/0	14/14/0	12/16/0	14/14/0	7/21/0	/

"W/T/L" represents how many instances MuSaGP significantly wins/ties/loses the comparison.

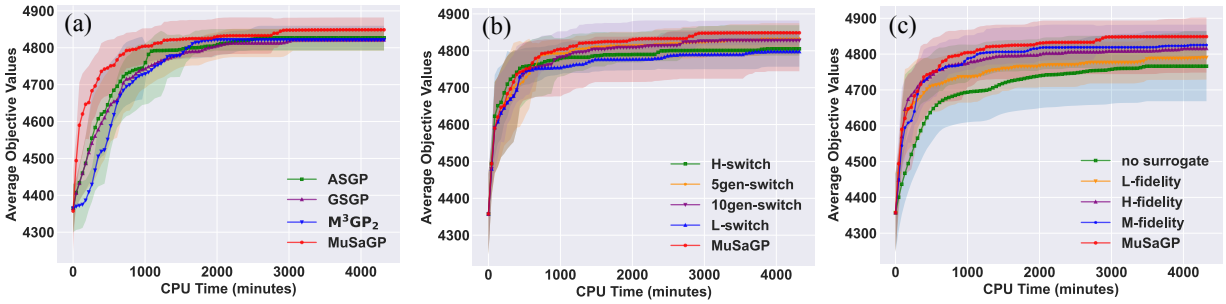

Fig. 7. Training curves of the best fitness values for different methods and 95% confidence interval: (a) Evaluates the performance against alternative surrogate GP approaches. (b) Compares MuSaGP variants employing diverse switching strategies. (c) Analyzes the impact of different surrogate selection techniques within MuSaGP variants.

and RL methods could achieve stable convergence within this specified time budget. The other parameters of the baselines are directly inherited from their original settings. Each algorithm is trained and tested 30 times independently on each instance. The experiments were carried out on a 64-bit CentOS Linux release 7.4. The hardware infrastructure features an Intel(R) Xeon(R) CPU E5-2695 v4 @ 2.10GHz.

C. Results and Discussions

We use the Wilcoxon rank sum test with Bonferroni correction for statistical analysis wherein the significance level is set to 0.05. In each table, (+), (-) and (=) indicate that MuSaGP performed statistically significantly better than, worse than, or comparable to the compared algorithm.

1) Comparison with Reactive Heuristics: The comparison results are shown in Table IV, including the mean value and standard deviation of evaluation metrics, and the Wilcoxon rank-sum test results with the significance level of 0.05. Overall, Table IV manifests that MuSaGP has significantly better performance than the baselines in most instances. The average gap of MuSaGP to the compared algorithm is calculated as $(f_{\mathrm{MuSaGP}} - f_{\mathrm{baseline}})/f_{\mathrm{baseline}}$, where f_{MuSaGP} and f_{baseline} represent the objective value achieved by MuSaGP and the compared baselines, respectively. Specifically, the average gaps to the baselines are 14.11% (D* Lite), 8.69% (CCRP), 26.77% (CASPER), 51.61% (MUEGF) and 8.70% (CASA). This suggests that MuSaGP is capable of significantly increasing air traffic network throughput by scheduling more aircraft within the same time duration compared with these other methods.

2) Comparison with Real Schedules: Table IV displays the air traffic network throughput for each instance, based on historical flight plans, termed 'REAL'. A comparison with these real schedules shows a substantial improvement in throughput when using MuSaGP, with an average gap of 27.20%. Furthermore, MuSaGP consistently outperforms the real schedules across all instances compared. This indicates that MuSaGP is capable of generating more efficient scheduling policies than the manually developed flight schedules typically recommended by air traffic controllers.

3) Comparison with Automated Scheduling Heuristics: The convergence curves of the compared GPs are shown in Fig. 7 (a). It shows that MuSaGP has the best search ability in the heuristic space since it keeps a faster growth than other GP methods. The comparison results are presented in the last four columns of Table IV. It can be observed that MuSaGP has significantly outperformed the four state-of-the-art automated heuristic design methods on most instances with the average gaps of 1.25% (ASGP), 1.09% (GSGP), 1.47% (M^3GP_2) and 0.71% (RLIPA). Besides, MuSaGP consistently outperforms the compared algorithms in terms of mean objective values across most instances,

TABLE V
MEAN, STANDARD DEVIATION, AND SIGNIFICANCE TEST OF THE 30 RUNS OF THE COMPARED ALGORITHMS ON THE **OBJECTIVE VALUES**. HIGHER VALUES INDICATE BETTER ALGORITHM PERFORMANCE. THE BEST RESULTS FOR EACH INSTANCE ARE HIGHLIGHTED IN BOLD. 6 OUT OF 28 TEST INSTANCES ARE PRESENTED. THE FULL RESULTS CAN BE FOUND IN THE SUPPLEMENTARY MATERIAL

Instance	H-switch	10gen-switch	L-switch	5gen-switch	no surrogate	H-fidelity	M-fidelity	L-fidelity	MuSaGP
mild-m1	1943.5(113)(+)	1950.5(83)(=)	1976.3(21)(=)	1986.1(24)(=)	1976.0(29)(=)	1950.9(83)(=)	1986.3(18)(=)	1997.5(34)(=)	**2002.0**(35)
mild-l1	3352.2(181)(+)	3374.8(128)(=)	3408.9(40)(+)	3415.9(47)(=)	3403.5(35)(+)	3371.4(136)(+)	3422.1(37)(=)	3424.1(54)(=)	**3448.2**(45)
moderate-m1	1982.3(118)(+)	1990.5(85)(=)	2017.9(26)(=)	2028.3(23)(=)	2016.3(23)(+)	1993.0(82)(+)	2028.2(25)(=)	2035.6(29)(=)	**2040.9**(27)
moderate-l1	3493.6(206)(=)	3507.1(132)(=)	3544.1(35)(=)	3553.7(44)(=)	3543.6(31)(=)	3508.5(138)(=)	3559.5(36)(=)	3558.7(50)(=)	**3578.2**(40)
severe-m1	1603.5(84)(=)	1608.0(68)(=)	1627.1(36)(=)	1652.4(31)(=)	1628.2(29)(=)	1605.2(61)(+)	1635.4(38)(=)	**1659.3**(33)(=)	1657.3(43)
severe-l1	2223.7(142)(+)	2241.5(105)(+)	2275.9(23)(=)	2281.3(25)(=)	2273.2(24)(+)	2246.1(104)(=)	2282.7(25)(=)	2284.5(29)(=)	**2296.3**(23)
W/T/L	10/18/0	8/20/0	13/15/0	5/22/1	19/9/0	10/18/0	9/19/0	13/15/0	/

achieving superior results in 26 out of 28 cases.

4) Running Time Comparison: The proposed MuSaGP algorithm is evaluated against three types of baselines: 1) Manually Designed Heuristics (D* Lite, CCRP, CASPER, MUEGF, CASA). These heuristics are applied directly to test instances, resulting only in online test running times. While they avoid training costs, their manual design is time-consuming and requires extensive expertise. 2) Expert-Assigned Schedules (REAL): This category includes historical flight plans crafted by air traffic experts for DATFM. These schedules have no measurable running time. 3) Automatic Heuristic Design Methods (ASGP, GSGP, M^3GP_2). These methods involve a training phase on training instances, followed by the application of the best-performing heuristics to test instances. They incur both offline training and online testing runtime. To ensure a fair comparison, all automatic heuristic design methods are given the same CPU runtime of 4320 minutes for training. Consequently, while the offline training time is uniform, online running times may vary. The full comparisons of runtime can be found in Table S2 of the supplementary material. The results show that MuSaGP has the fastest runtime among the compared algorithms. Its average runtime is 1.51 seconds versus 1.62 seconds of the second-fast algorithm M^3GP_2. This fast response ability makes it well-suited for addressing the real-time demands of DATFM in practice.

D. Component Analysis

To verify the effectiveness of each component of the newly proposed MuSaGP, we designed a set of control experiments on the instances.

1) Effects of Adaptive Switching Strategy: To evaluate the effectiveness of the adaptive switching strategy in MuSaGP, four variants with distinct switching frequencies were developed.

- **H-switch:** MuSaGP employs standard surrogate management strategies used in existing surrogate GPs. It alternates between surrogate and exact evaluation every generation, resulting in the **h**ighest switching frequency.
- **L-switch:** The algorithm consistently uses the surrogate from the initial population without ever switching to exact evaluation, making this the variant with the **l**owest switching frequency.
- **10gen-switch/5gen-switch** The algorithm switches to one generation of exact evolution after every 10/5 generations of surrogate evolution.

The variants are given the same training time of 4320 minutes and the convergence curves are depicted in Fig. 7 (b). The comparison results are presented in Table V. It shows that MuSaGP performs better than the four variants in most instances with an average gap of 1.08%. This indicates the effectiveness of the newly proposed adaptive switching strategy. Additionally, from the statistical significance test, it is observed that the rank of the variants is 5gen-switch $>$ 10gen-switch $>$ H-switch $>$ L-switch. Switching from surrogate to exact evolution allows the algorithm to obtain the latest information from the evolved population. Conversely, switching from exact to surrogate evolution enables exploration of a broader heuristic space within the same time frame, as surrogate evaluations are less resource-intensive than exact evaluations. This is why variants with dynamic strategies outperform those without switching strategies. Unlike variants with predetermined switching intervals, MuSaGP adaptively determines the switching points based on stagnation detection, contributing to its superior performance among the compared variants.

2) Effects of Multi-fidelity Surrogates: To evaluate the effectiveness of the multi-fidelity surrogates in MuSaGP, four variants with different surrogate managements were developed.

- **no surrogate:** In this variant, surrogates are completely removed from MuSaGP, transforming it into a basic GP model.
- **H/M/L-fidelity:** These variants of MuSaGP utilize only a single surrogate at high, medium, or low fidelity levels. The respective simulation timespans are set at $3 \cdot T_{\max}/4$, $T_{\max}/2$ and $T_{\max}/4$.

The convergence curves of the compared variants are depicted in Fig. 7 (c). This indicates that the variants using surrogates achieve significantly faster convergence compared with the variants without surrogates. The comparison results are presented in Table V. Overall, MuSaGP has better performance than the compared variants. The average gaps to the variants are 2.44% (no surrogate), 2.18% (H-fidelity), 1.04% (M-fidelity), and 0.53% (L-fidelity), respectively. Within the same training timeframe, the H-fidelity variant more effec-

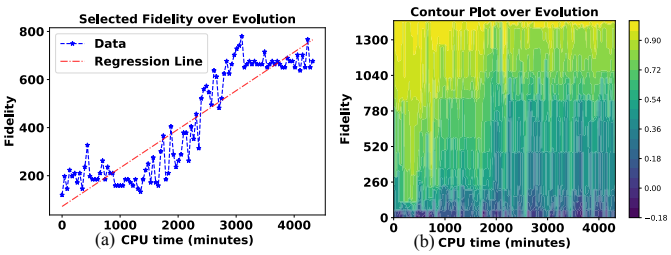

Fig. 8. Analysis of the utilized surrogates: (a) The fidelity of the selected surrogates during the evolutionary process; (b) Accuracy heatmap of surrogates across fidelity and CPU time.

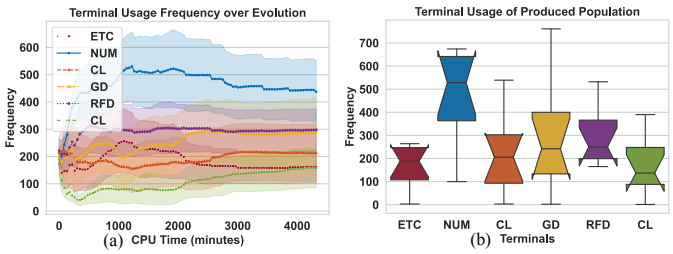

Fig. 9. (a) The usage frequency of the six terminals during the evolutionary process. (b) The usage frequency of the terminals in the final population.

tively differentiates individuals but is limited in exploration due to the higher cost of evaluating high-fidelity surrogates. On the other hand, the L-fidelity variant, while capable of exploring broader regions, is hindered by its low-fidelity surrogates, which may lead to misleading search directions. MuSaGP achieves a balance between surrogates of varying fidelities by selecting the most appropriate surrogate for each situation, effectively trading off between simulation speed and accuracy. It leverages the strengths of different fidelity surrogates to maximize overall performance.

In summary, the ablation study effectively validates the effectiveness of both the adaptive switching strategy and the multi-fidelity surrogates in MuSaGP.

V. Further Analysis

A. Utilizing Surrogates in Evolutionary Process

To demonstrate the fidelity of surrogates used by MuSaGP throughout the evolutionary process (with exact evaluation considered as the highest fidelity surrogate), we plotted the average fidelity across 30 independent runs. The fidelity is represented by the simulation timespan T^*. A higher value of T^* denotes a higher fidelity of the surrogate. This is shown in Fig. 8 (a). The data reveals a clear trend of MuSaGP increasingly selecting higher fidelity surrogates as the evolution progresses. Additionally, the regression analysis of the collected data yields a goodness of fit value of 0.82.

To understand why MuSaGP increasingly selects higher fidelity surrogates during evolution, we analyzed the accuracy (Spearman correlation) of surrogates at different fidelity levels over the evolutionary process, as shown in Fig. 8 (b). Initially, low/medium fidelity surrogates exhibit high accuracy, but their accuracy diminishes over time. By the final stages, only high-fidelity surrogates maintain strong accuracy. This decline in accuracy for lower fidelity surrogates is due to their inability to effectively differentiate between individuals as the population converges on promising regions in the heuristic space. Consequently, these surrogates may lead to misleading search directions and population stagnation. Therefore, MuSaGP increasingly relies on high fidelity surrogates or exact evaluations towards the end of the evolutionary process to accurately distinguish superior individuals.

B. Frequency of Terminal Usage

To investigate the importance of the developed terminals on the evolved scheduling policy, we tracked the usage frequency of different terminals within the population throughout the evolutionary process. For instance, if an individual is expressed as $\max\{\mathsf{ETC}, \mathsf{FDL}\} + \mathsf{ETC}$, the frequencies of ETC and FDL would be counted as 2 and 1, respectively. Fig. 9 shows the usage trends over time and the prevalence of each terminal in the final population. Initially, all six terminals are used with similar frequency. However, as evolution progresses, NUM and GD become the most prominent, indicating their importance. Given that NUM signifies the number of aircraft anticipated in the future and GD represents the geographical distance to the next waypoint, their frequent use underscores a balancing act between current route length and expected future traffic density.

C. Semantic Analysis of Evolved Policies

To gain a further understanding of the behavior of the scheduling policies, a representative scheduling policy is selected. The scheduling policy is shown in Fig. 10. To interpret the routing policy evolved by MuSaGP, we rewrite it by the following equations:

$$\mathsf{SP} = \mathsf{S}_A + \mathsf{S}_B + \mathsf{S}_C \tag{15}$$

$$\mathsf{S}_A = \mathsf{ETC} * (2\mathsf{GD} - \mathsf{ETC}) - \mathsf{GD} * \mathsf{GD} * \mathsf{FDL} \tag{16}$$

$$\mathsf{S}_B = \mathsf{CL} * \mathsf{CL} * \mathsf{RFD} * (\mathsf{RFD} - \mathsf{GD})/\mathsf{FDL} \tag{17}$$

$$\mathsf{S}_C = \mathsf{CL} - \mathsf{NUM} * \mathsf{NUM} \tag{18}$$

We can identify the following patterns and interpretations from the above rule set.

- For S_A: If $2\mathsf{GD} > \mathsf{ETC}$, indicating less congested, sparser airspace, the policy prefers nodes with a shorter geographical distance (a smaller GD). Conversely, if $2\mathsf{GD} < \mathsf{ETC}$, suggesting congested airspace with high estimated time cost between nodes, the policy favors nodes linked by less congested routes (smaller ETC), considering the quadratic term.
- For S_B: The policy initially prioritizes nodes with higher segment capacity (larger CL). When $\mathsf{RFD} > \mathsf{GD}$, it indicates that the aircraft are far from the destination airport, leading the policy to favor nodes with a smaller remaining flight distance (a smaller

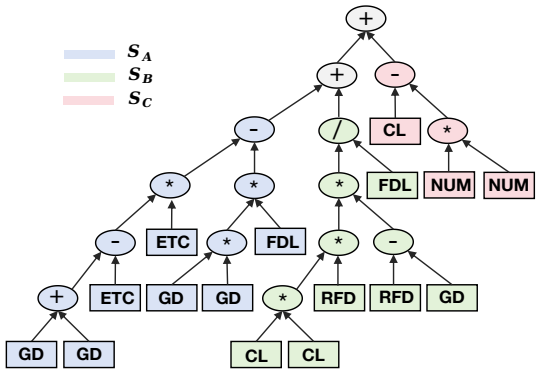

Fig. 10. The tree structure of the evolved scheduling policy by MuSaGP

RFD) due to the quadratic term. Conversely, if RFD $<$ GD, suggesting that the aircraft are approaching the destination, the policy tends to select nodes with a less flow density in the link (a smaller FDL).

- For S_C: This term indicates that the policy consistently favors nodes with higher segment capacity (larger CL) and those expected to encounter fewer aircraft in the future (smaller NUM).

VI. Conclusion and Future Work

This article aimed to develop a surrogate-assisted genetic programming approach named MuSaGP to automatically evolve scheduling heuristics in large-scale dynamic air traffic flow management (DATFM). This goal was achieved by designing six problem-specific terminals and an effective low-level heuristic template, coupled with a novel strategy for managing multi-fidelity surrogates. This approach intelligently determined the timing and selection of surrogates throughout the evolutionary process. The results indicated that the proposed MuSaGP significantly outperformed the compared state-of-the-art methods in various scenarios, differing in scale and conditions. MuSaGP contributes to the field of Air Traffic Management (ATM) by providing an automated and scalable solution for evolving effective and interpretable reactive heuristics in dynamic and large-scale environments. In addition, the component analysis revealed that the surrogate management strategy effectively enhances the search capabilities of Surrogate-assisted Genetic Programming (SaGP) within the heuristic space. It provides SaGP with a new adaptive strategy that allows for a more efficient management of computational resources.

Future research could build on these findings by exploring new directions, including the following.

1) Further Reducing Training Time. Although the surrogates have improved the training efficiency, the offline time budget over more training instances is still high. A possible future direction is to investigate whether we could use only a few training instances to achieve similar performance, thus further reducing the offline computational demand.
2) Simultaneous Optimization of Multiple ATM Objectives. Beyond maximizing air traffic throughput (i.e., operational efficiency), it is feasible to concurrently optimize additional ATM objectives, such as fuel reduction, for economic benefits. Developing new multi-objective approaches to achieve this goal is highly desirable.

References

[1] R. Shone, K. Glazebrook, and K. G. Zografos, "Applications of stochastic modeling in air traffic management: Methods, challenges and opportunities for solving air traffic problems under uncertainty," *Eur. J. Oper. Res.*, vol. 292, no. 1, pp. 1–26, 2021.
[2] P. Brooker, "Sesar and nextgen: investing in new paradigms," *J. Navig.*, vol. 61, no. 2, pp. 195–208, 2008.
[3] O. Richard, S. Constans, and R. Fondacci, "Computing 4d near-optimal trajectories for dynamic air traffic flow management with column generation and branch-and-price," *Transp. Plan. Technol.*, vol. 34, no. 5, pp. 389–411, 2011.
[4] A. Mukherjee and M. Hansen, "Dynamic stochastic optimization model for air traffic flow management with en route and airport capacity constraints," in *Proc. 6th USA/Eur. Air Traffic Manage. R&D Seminar*, 2005.
[5] Y. Wang, C. Liu, H. Wang, and V. Duong, "Slot allocation for a multiple-airport system considering airspace capacity and flying time uncertainty," *Transport. Res. Part C Emerg. Technol.*, vol. 153, p. 104185, 2023.
[6] A. Mukherjee and M. Hansen, "A dynamic rerouting model for air traffic flow management," *Transport. Res. Part B Meth.*, vol. 43, no. 1, pp. 159–171, 2009.
[7] D. Bertsimas and S. S. Patterson, "The traffic flow management rerouting problem in air traffic control: A dynamic network flow approach," *Transp. Sci.*, vol. 34, no. 3, pp. 239–255, 2000.
[8] K. Ng, C. Lee, F. T. Chan, and Y. Qin, "Robust aircraft sequencing and scheduling problem with arrival/departure delay using the min-max regret approach," *Transport. Res. Part E: Logist. Transport. Rev.*, vol. 106, pp. 115–136, 2017.
[9] M. Kapolke, N. Fürstenau, A. Heidt, F. Liers, M. Mittendorf, and C. Weiß, "Pre-tactical optimization of runway utilization under uncertainty," *J. Air Transp. Manag.*, vol. 56, pp. 48–56, 2016.
[10] B. Hao, K. Cai, Y.-P. Fang, A. Fadil, and D. Feng, "A moment-based distributionally robust optimization model for air traffic flow management," in *2021 2021 IEEE/AIAA 40th DASC*. IEEE, 2021, pp. 1–7.
[11] J. Chen, L. Chen, and D. Sun, "Air traffic flow management under uncertainty using chance-constrained optimization," *Transport. Res. Part B Meth.*, vol. 102, pp. 124–141, 2017.
[12] J. Chen and D. Sun, "Stochastic ground-delay-program planning in a metroplex," *J. Guid. Control Dyn.*, vol. 41, no. 1, pp. 231–239, 2018.
[13] G. G. N. Sandamali, R. Su, K. L. K. Sudheera, and Y. Zhang, "A safety-aware real-time air traffic flow management model under demand and capacity uncertainties," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 8615–8628, 2021.
[14] A. Khassiba, F. Bastin, S. Cafieri, B. Gendron, and M. Mongeau, "Two-stage stochastic mixed-integer programming with chance constraints for extended aircraft arrival management," *Transp. Sci.*, vol. 54, no. 4, pp. 897–919, 2020.
[15] S. Vaidhun, Z. Guo, J. Bian, H. Xiong, and S. K. Das, "Dynamic path planning for unmanned aerial vehicles under deadline and sector capacity constraints," *IEEE Trans. Emerg. Topics Comput.*, vol. 6, no. 4, pp. 839–851, 2021.
[16] D. García-Heredia, E. Molina, M. Laguna, and A. Alonso-Ayuso, "A solution method for the shared resource-constrained multi-shortest path problem," *Expert Syst. Appl.*, vol. 182, p. 115193, 2021.
[17] N. Ivanov, F. Netjasov, R. Jovanović, S. Starita, and A. Strauss, "Air traffic flow management slot allocation to minimize propagated delay and improve airport slot adherence," *Transport. Res. Part A Policy Pract.*, vol. 95, pp. 183–197, 2017.
[18] T. J. Callantine, "Cats-based air traffic controller agents," Tech. Rep., 2002, https://ntrs.nasa.gov/citations/20030005106.

[19] M. Freed, "Reactive prioritization," in *Proc. 2nd NASA Int. Workshop Plan. Sched. Space*. National Aeronautics and Space Administration Washington, DC, 2000.

[20] T. Kistan, A. Gardi, R. Sabatini, S. Ramasamy, and E. Batuwangala, "An evolutionary outlook of air traffic flow management techniques," *Prog.Aerosp. Sci.*, vol. 88, pp. 15–42, 2017.

[21] Eurocontrol, "ATFCM Operations Manual - Network Manager," EUROCONTROL, Tech. Rep., 2021, https://www.eurocontrol.int/sites/default/files/2021-01/eurocontrol-atfcm-operations-manual-24-1-18012021.pdf.

[22] F. Zhang, S. Nguyen, Y. Mei, and M. Zhang, *Genetic Programming for Production Scheduling*. Springer, 2021.

[23] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Survey on genetic programming and machine learning techniques for heuristic design in job shop scheduling," *IEEE Trans. Evol. Comput.*, vol. 28, no. 1, pp. 147–167, 2023.

[24] S. Wang, Y. Mei, M. Zhang, and X. Yao, "Genetic programming with niching for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 26, no. 1, pp. 73–87, 2021.

[25] G. Gao, Y. Mei, B. Xin, Y.-H. Jia, and W. N. Browne, "Automated coordination strategy design using genetic programming for dynamic multipoint dynamic aggregation," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 13 521–13 535, 2021.

[26] Y. Mei, Q. Chen, A. Lensen, B. Xue, and M. Zhang, "Explainable artificial intelligence by genetic programming: A survey," *IEEE Trans. Evol. Comput.*, vol. 27, no. 3, pp. 621–641, 2022.

[27] X.-C. Liao, Y.-H. Jia, X.-M. Hu, and W.-N. Chen, "Uncertain commuters assignment through genetic programming hyper-heuristic," *IEEE Trans. Comput. Soc. Syst.*, vol. 11, no. 2, pp. 2606–2619, 2023.

[28] P. Volf, D. Šišlák, and M. Pěchouček, "Large-scale high-fidelity agent-based simulation in air traffic domain," *Cybern Syst*, vol. 42, no. 7, pp. 502–525, 2011.

[29] S. Nguyen, M. Zhang, and K. C. Tan, "Surrogate-assisted genetic programming with simplified models for automated design of dispatching rules," *IEEE Trans. Cybern.*, vol. 47, no. 9, pp. 2951–2965, 2016.

[30] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Collaborative multifidelity-based surrogate models for genetic programming in dynamic flexible job shop scheduling," *IEEE Trans. Cybern.*, vol. 52, no. 8, pp. 8142–8156, 2022.

[31] F. Zhang, Y. Mei, and M. Zhang, "Surrogate-assisted genetic programming for dynamic flexible job shop scheduling," in *Proc. Aust. Joint Conf. Artif. Intell.* Springer, 2018, pp. 766–772.

[32] Y. Zeiträg, J. R. Figueira, N. Horta, and R. Neves, "Surrogate-assisted automatic evolving of dispatching rules for multi-objective dynamic job shop scheduling using genetic programming," *Expert Syst. Appl*, vol. 209, p. 118194, 2022.

[33] H. Spiess, "Conical volume-delay functions," *Transp Sci.*, vol. 24, no. 2, pp. 153–158, 1990.

[34] J. Mitchell, V. Polishchuk, and J. Krozel, "Airspace throughput analysis considering stochastic weather," in *AIAA Guid., Nav., and Control Conf. and Exhib.*, 2006, p. 6770.

[35] O. Richetta and A. R. Odoni, "Dynamic solution to the ground-holding problem in air traffic control," *Transport. Res. Part A Policy Pract.*, vol. 28, no. 3, pp. 167–185, 1994.

[36] A. Agustı, A. Alonso-Ayuso, L. F. Escudero, C. Pizarro *et al.*, "On air traffic flow management with rerouting. part ii: Stochastic case," *Eur. J. Oper. Res.*, vol. 219, no. 1, pp. 167–177, 2012.

[37] Y. Liu, Y. Mei, M. Zhang, and Z. Zhang, "A predictive-reactive approach with genetic programming and cooperative coevolution for the uncertain capacitated arc routing problem," *Evol. Comput.*, vol. 28, no. 2, pp. 289–316, 2020.

[38] M. Prais and C. C. Ribeiro, "Reactive grasp: An application to a matrix decomposition problem in tdma traffic assignment," *INFORMS J. Comput.*, vol. 12, no. 3, pp. 164–176, 2000.

[39] S. Vaidhun, Z. Guo, J. Bian, H. Xiong, and S. K. Das, "Priority-based multi-flight path planning with uncertain sector capacities," in *2020 12th Int. Conf. Adv. Comput. Intell. (ICACI)*. IEEE, 2020, pp. 529–535.

[40] A. Guitart, D. Delahaye, F. M. Camino, and E. Feron, "Collaborative generation of local conflict free trajectories with weather hazards avoidance," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12 831–12 842, 2023.

[41] S. Wang, Y. Mei, M. Zhang, and X. Yao, "Genetic programming with niching for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 26, no. 1, pp. 73–87, 2021.

[42] S. Wang, Y. Mei, and M. Zhang, "A multi-objective genetic programming algorithm with α dominance and archive for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 27, no. 6, pp. 1633–1647, 2022.

[43] X.-C. Liao, W.-N. Chen, Y.-H. Jia, and W.-J. Qiu, "Towards scalable dynamic traffic assignment with streaming agents: a decentralized control approach using genetic programming," *IEEE Trans. Emerg. Topics Comput.*, vol. 8, no. 1, pp. 942–955, 2024.

[44] M. O'Neill, "Riccardo poli, william b. langdon, nicholas f. mcphee: A field guide to genetic programming: Lulu. com, 2008, 250 pp, isbn 978-1-4092-0073-4," 2009.

[45] J. Branke, S. Nguyen, C. W. Pickardt, and M. Zhang, "Automated design of production scheduling heuristics: A review," *IEEE Trans. Evol. Comput.*, vol. 20, no. 1, pp. 110–124, 2015.

[46] M. A. Ardeh, Y. Mei, and M. Zhang, "Genetic programming with knowledge transfer and guided search for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, vol. 26, no. 4, pp. 765–779, 2021.

[47] T. Hildebrandt and J. Branke, "On using surrogates with genetic programming," *Evol. Comput.*, vol. 23, no. 3, pp. 343–367, 2015.

[48] F. Zhang, Y. Mei, S. Nguyen, M. Zhang, and K. C. Tan, "Surrogate-assisted evolutionary multitask genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, vol. 25, no. 4, pp. 651–665, 2021.

[49] J. Branke, M. Asafuddoula, K. S. Bhattacharjee, and T. Ray, "Efficient use of partially converged simulations in evolutionary optimization," *IEEE Trans. Evol. Comput.*, vol. 21, no. 1, pp. 52–64, 2016.

[50] M. M. Mamun, H. K. Singh, and T. Ray, "A multifidelity approach for bilevel optimization with limited computing budget," *IEEE Trans. Evol. Comput.*, vol. 26, no. 2, pp. 392–399, 2021.

[51] A. Kenny, T. Ray, and H. K. Singh, "An iterative two-stage multi-fidelity optimization algorithm for computationally expensive problems," *IEEE Trans. Evol. Comput.*, vol. 27, no. 3, pp. 520–534, 2022.

[52] A. Habib, H. K. Singh, and T. Ray, "A multiple surrogate assisted multi/many-objective multi-fidelity evolutionary algorithm," *Inf. Sci.*, vol. 502, pp. 537–557, 2019.

[53] T. Helmuth, L. Spector, and J. Matheson, "Solving uncompromising problems with lexicase selection," *IEEE Trans. Evol. Comput.*, vol. 19, no. 5, pp. 630–643, 2014.

[54] M. Xu, Y. Mei, F. Zhang, and M. Zhang, "Genetic programming with lexicase selection for large-scale dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, pp. 1–1, 2023, doi=10.1109/TEVC.2023.3244607.

[55] F. Zhang, Y. Mei, S. Nguyen, K. C. Tan, and M. Zhang, "Instance rotation based surrogate in genetic programming with brood recombination for dynamic job shop scheduling," *IEEE Trans. Evol. Comput.*, vol. 27, no. 5, pp. 1192–1206, 2022.

[56] R. F. Woolson, "Wilcoxon signed-rank test," *Wiley encyclopedia of clinical trials*, pp. 1–3, 2007.

[57] O. L. O. Astivia and B. D. Zumbo, "Population models and simulation methods: The case of the spearman rank correlation," *Br. J. Math. Stat. Psychol.*, vol. 70, no. 3, pp. 347–367, 2017.

[58] K. Deb and S. Gupta, "Understanding knee points in bicriteria problems and their implications as preferred solution principles," *Eng. Optim.*, vol. 43, no. 11, pp. 1175–1204, 2011.

[59] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: Nsga-ii," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Aug., 2002.

[60] S. Reitmann, S. Alam, and M. Schultz, "Advanced quantification of weather impact on air traffic management," in *ATM Seminar*, vol. 44, 2019, pp. 554–567.

[61] K. Shahabi and J. P. Wilson, "Casper: Intelligent capacity-aware evacuation routing," *Comput Environ Urban Syst.*, vol. 46, pp. 12–24, 2014.

[62] M. A. Dulebenets, J. Pasha, O. F. Abioye, M. Kavoosi, E. E. Ozguven, R. Moses, W. R. Boot, and T. Sando, "Exact and heuristic solution algorithms for efficient emergency evacuation in areas with vulnerable populations," *Int. J. Disaster Risk Reduct.*, vol. 39, p. 101114, 2019.

[63] Y. Wang, W. Cai, Y. Tu, and J. Mao, "Reinforcement-learning-informed prescriptive analytics for air traffic flow management," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 4188–4202, 2024.

[64] M. Hollander, D. A. Wolfe, and E. Chicken, *Nonparametric statistical methods*. John Wiley & Sons, 2013.